Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 8

Aim:

To find the minimum-cost route that lets a SwiftShip delivery truck visit every city exactly once and return to the starting city (Travelling Salesman Problem), by implementing the Least Cost (LC) Branch and Bound algorithm with cost matrix reduction to compute lower bounds and prune unpromising partial routes.

Theory:

1. The problem is the Travelling Salesman Problem (TSP): find the minimum-cost cycle that visits each city exactly once and returns to the start. Since there are (N-1)! possible tours, brute force is infeasible for large N.
2. Branch and Bound explores partial solutions (paths) as nodes of a search tree. Each node has a partial path, the cost so far, and a lower bound on the cost of completing the tour (computed by matrix reduction).
3. Nodes are expanded in increasing order of lower bound (best-first, Least Cost). If the lower bound of a node is greater than or equal to the cost of the best tour found so far, the node is pruned.
4. Matrix reduction: impossible edges are set to infinity; the minimum of each row is subtracted from that row and the minimum of each column is subtracted from that column. The sum of all subtractions is the lower bound contribution.
5. LC Branch and Bound with the reduced matrix is exact and usually prunes a very large part of the search tree compared with brute force.

Algorithm (LC Branch and Bound)

1. Build the N x N cost matrix (cost[i][j] = fuel cost from i to j; diagonal = infinity).
2. Reduce the matrix and compute the initial lower bound lb0.
3. Create the root node with the reduced matrix, lb = lb0, path = [start city] and level = 0.
4. Use a priority queue (min-heap) keyed by lb; pop the node with the smallest lb.
5. If the node has visited all cities (level = N-1), update the best tour and cost.
6. Otherwise, for each city j not in the path, create a child: copy the parent matrix, set the row of the parent city and the column of j to infinity, block the edge back to the start, and reduce the matrix. child.lb = parent.lb + cost[parent][j] + reduction cost.
7. If child.lb < best cost, push the child into the queue; otherwise prune it.
8. Continue until the queue is empty. The best solution found is the optimal route.

Complexity Analysis

- Best case: pruning is so effective that very few nodes are examined.
- Worst case: factorial time O(N!), when no pruning is possible.
- Bounding cost per node: O(N^2) to copy and reduce the matrix.

- Practical observation: N of about 10 to 15 cities can be solved exactly within seconds; for larger N the run time grows quickly.
- Time constraints can be handled by adding penalties for late arrival or by rejecting paths that violate time windows.

Code:

```
import java.util.*;

public class TSPBranchBound {
    static final int INF = Integer.MAX_VALUE / 4;
    static int N;
    static long nodesGenerated, nodesPruned;

    static class Node implements Comparable<Node> {
        int[][] matrix;
        int lb;
        List<Integer> path;
        int level;
        int city;

        Node(int[][] matrix, int lb, List<Integer> path, int level, int city) {
            this.matrix = matrix;
            this.lb = lb;
            this.path = path;
            this.level = level;
            this.city = city;
        }

        public int compareTo(Node o) {
            return Integer.compare(this.lb, o.lb);
        }
    }

    static int[][] copy(int[][] m) {
        int[][] c = new int[m.length][];
        for (int i = 0; i < m.length; i++) c[i] = m[i].clone();
        return c;
    }

    // Row/column reduction. Modifies m and returns the total amount subtracted.
    static int reduce(int[][] m) {
        int total = 0;
        for (int i = 0; i < N; i++) {
            int min = INF;
            for (int j = 0; j < N; j++) min = Math.min(min, m[i][j]);
            if (min != INF && min > 0) {
                total += min;
                for (int j = 0; j < N; j++) if (m[i][j] != INF) m[i][j] -= min;
            }
        }
        for (int j = 0; j < N; j++) {
            int min = INF;
```

```
            for (int i = 0; i < N; i++) min = Math.min(min, m[i][j]);
            if (min != INF && min > 0) {
                total += min;
                for (int i = 0; i < N; i++) if (m[i][j] != INF) m[i][j] -= min;
            }
        }
        return total;
    }

    static int bestCost;
    static List<Integer> bestPath;

    static void solve(int[][] cost) {
        N = cost.length;
        nodesGenerated = 0;
        nodesPruned = 0;
        bestCost = INF;
        bestPath = null;

        int[][] root = copy(cost);
        for (int i = 0; i < N; i++) root[i][i] = INF;
        int lb0 = reduce(root);

        PriorityQueue<Node> pq = new PriorityQueue<>();
        List<Integer> p0 = new ArrayList<>();
        p0.add(0);
        pq.add(new Node(root, lb0, p0, 0, 0));
        nodesGenerated++;

        while (!pq.isEmpty()) {
            Node node = pq.poll();
            if (node.lb >= bestCost) { nodesPruned++; continue; }

            if (node.level == N - 1) {          // all cities visited, lb is the exact
tour cost
                bestCost = node.lb;
                bestPath = new ArrayList<>(node.path);
                bestPath.add(0);
                continue;
            }

            int u = node.city;
            for (int v = 0; v < N; v++) {
                if (node.path.contains(v)) continue;
                int[][] m = copy(node.matrix);
                int edge = m[u][v];
                for (int k = 0; k < N; k++) { m[u][k] = INF; m[k][v] = INF; }
                if (node.level + 1 < N - 1) m[v][0] = INF;    // block premature
return to start
                int red = reduce(m);
                int childLb = node.lb + edge + red;
                nodesGenerated++;
                if (childLb < bestCost) {
                    List<Integer> np = new ArrayList<>(node.path);
```

```
                np.add(v);
                pq.add(new Node(m, childLb, np, node.level + 1, v));
            } else {
                nodesPruned++;
            }
        }
    }
}

// Brute force for verification: all (N-1)! permutations
static int bruteBest;
static void permute(int[][] cost, int[] perm, int k, int cur) {
    if (cur >= bruteBest) return;
    if (k == N) {
        bruteBest = Math.min(bruteBest, cur + cost[perm[N - 1]][0]);
        return;
    }
    for (int i = k; i < N; i++) {
        int t = perm[k]; perm[k] = perm[i]; perm[i] = t;
        permute(cost, perm, k + 1, cur + cost[perm[k - 1]][perm[k]]);
        t = perm[k]; perm[k] = perm[i]; perm[i] = t;
    }
}

static int bruteForce(int[][] cost) {
    int n = cost.length;
    int[] perm = new int[n];
    for (int i = 0; i < n; i++) perm[i] = i;
    bruteBest = INF;
    permute(cost, perm, 1, 0);
    return bruteBest;
}

static long factorial(int n) {
    long f = 1;
    for (int i = 2; i <= n; i++) f *= i;
    return f;
}

static String pathString(List<Integer> p) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < p.size(); i++) {
        if (i > 0) sb.append(" -> ");
        sb.append(p.get(i));
    }
    return sb.toString();
}

public static void main(String[] args) {
    int I = INF;
    int[][] cost = {
            {I, 20, 30, 10, 11},
            {15, I, 16, 4, 2},
            {3, 5, I, 2, 4},
```

```
                {19, 6, 18, I, 3},
                {16, 4, 7, 16, I}
        };

        System.out.println("=== Test 1: 5 cities (fuel-cost matrix, City 0 = depot)
===");
        System.out.println("Cost matrix:");
        for (int[] row : cost) {
            StringBuilder sb = new StringBuilder();
            for (int x : row) sb.append(String.format("%5s", x >= INF ? "INF" :
String.valueOf(x)));
            System.out.println(sb);
        }
        solve(cost);
        System.out.println("\nLC Branch & Bound result:");
        System.out.println("  Minimum tour cost : " + bestCost);
        System.out.println("  Optimal route     : " + pathString(bestPath));
        System.out.println("  Nodes generated   : " + nodesGenerated + ", pruned : " +
nodesPruned);
        System.out.println("  Brute-force check : " + bruteForce(cost) + "
(permutations = " + factorial(N - 1) + ")");

        System.out.println("\n=== Test 2: 10 random cities (seed = 1) ===");
        int n = 10;
        Random rnd = new Random(1);
        int[][] big = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) big[i][j] = (i == j) ? INF : 10 +
rnd.nextInt(90);
        solve(big);
        System.out.println("LC Branch & Bound result:");
        System.out.println("  Minimum tour cost : " + bestCost);
        System.out.println("  Optimal route     : " + pathString(bestPath));
        System.out.println("  Nodes generated   : " + nodesGenerated + ", pruned : " +
nodesPruned);
        System.out.println("  Brute-force nodes would be about (N-1)! = " +
factorial(n - 1) + " complete tours");
        System.out.println("  Brute-force check : " + bruteForce(big));
    }
}
```

Output:

```
=== Test 1: 5 cities (fuel-cost matrix, City 0 = depot) ===
Cost matrix:
  INF   20   30   10   11
   15  INF   16    4    2
    3    5  INF    2    4
   19    6   18  INF    3
   16    4    7   16  INF

LC Branch & Bound result:
  Minimum tour cost : 28
  Optimal route     : 0 -> 3 -> 1 -> 4 -> 2 -> 0
```

```
  Nodes generated   : 11, pruned : 6
  Brute-force check : 28  (permutations = 24)

=== Test 2: 10 random cities (seed = 1) ===
LC Branch & Bound result:
  Minimum tour cost : 254
  Optimal route     : 0 -> 6 -> 7 -> 5 -> 1 -> 9 -> 8 -> 3 -> 4 -> 2 -> 0
  Nodes generated   : 60, pruned : 48
  Brute-force nodes would be about (N-1)! = 362880 complete tours
  Brute-force check : 254
```

Conclusion:

The LC Branch and Bound algorithm found the optimal tour of cost 28 for the 5-city example (City 0 -> 3 -> 1 -> 4 -> 2 -> 0) by generating only 11 nodes, and its answer matched the brute-force result. On a 10-city random instance it generated just 60 nodes and pruned 48, instead of examining all 362,880 possible tours, and again matched the brute-force cost of 254. The worst-case time is still O(N!) and each node costs O(N^2) for matrix reduction, so the method is practical for a moderate number of cities. For larger route sets, it can be combined with heuristics such as 2-opt or Christofides for scalability.